INLP Assignment 3

Yash More, 2024114004

Abstract

This report presents the implementation and evaluation of neural sequence models for three tasks: (1) decrypting a substitution cipher using RNN and LSTM, (2) language modeling with SSM for word-level next-word prediction and BiLSTM for word-level masked language modeling, and (3) using character-level language models to correct decryption errors in noisy cipher text. I've implemented all models from scratch in PyTorch and evaluated their performance using character accuracy, word accuracy, perplexity, and chrF scores. In Task 3, I show that character-level language modeling is highly effective at denoising cipher outputs compared to word-level approaches.

1 Task 1: Decrypting the Cipher

1.1 Architecture and Hyperparameters

RNN: I implement a single-layer vanilla RNN with 256-dimensional token embeddings and 512 hidden units, processing cipher tokens sequentially. The model applies 0.2 dropout for regularization and is trained with sequences of length 128, batch size 64, learning rate 0.001 for 50 epochs.

LSTM: I use a 2-layer stacked LSTM architecture with 256-dimensional embeddings, 512 hidden units per layer, and gating mechanisms (forget, input, output gates) to control information flow. Higher dropout of 0.3 compensates for the deeper architecture. Training uses longer sequences (256) and larger batches (128) with a cosine annealing LR schedule (lr_min=1e-5) over 100 epochs.

1.2 Results

Table 1 presents the performance metrics for both models evaluated on the clean cipher (cipher_00.txt).

Table 1: Task 1: Decryption Performance

Model	Test Loss	Char Acc	Word Acc	Levenshtein
RNN	1.680	0.523	0.225	269,268
LSTM	1.229	0.640	0.429	203,325

1.3 Analysis

Performance Comparison:

- **Character Accuracy:** LSTM achieves 64.0% vs RNN's 52.3% (11.7 percentage point improvement).
- **Word Accuracy:** LSTM achieves 42.9% vs RNN's 22.5% (20.4 percentage point improvement).
- **Strength of LSTM:** The forget gate enables selective retention of relevant information, the input gate controls the incorporation of new information, and the output gate regulates what is exposed to the next hidden state, while the cell state provides a stable pathway for gradient flow, effectively addressing the vanishing gradient problem and enabling modeling of long-range dependencies.
- **Limitations of LSTM:** LSTM incurs higher computational cost due to its gating mechanisms, both models struggle to achieve very high decryption accuracy (above 95%), and character-level modeling remains inherently more challenging compared to word-level approaches.

2 Task 2: Language Modeling

2.1 Architecture and Hyperparameters

SSM (Next Word Prediction): I implement a word-level State Space Model with 256-dimensional embeddings and 512 state size, using a causal left-to-right prediction approach where $s_t = \tanh(s_{t-1}A + x_tB)$. The model is trained on sequences of 256 words with batch size 256, learning rate 0.001 with cosine annealing to lr_min=1e-6.

BiLSTM (Masked Language Modeling): I use a bidirectional LSTM with 128-dimensional embeddings, 192 hidden units per direction, and forward-backward processing to capture context from both sides. The MLM training randomly masks 15% of tokens and predicts only masked positions. Training uses 256-word sequences, batch size 256, with weight decay 1e-4 and early stopping patience of 10 epochs.

2.2 Results

Table 2: Task 2: Language Modeling Performance (Word-Level)

Model	Test Loss	Perplexity
SSM (Next Word)	7.058	1161.61
BiLSTM (MLM)	6.886	978.63

2.3 Analysis

Performance Comparison:

- BiLSTM achieves 15.8% lower perplexity than SSM
- Both models show high perplexity, indicating word-level modeling difficulty

Why BiLSTM Performs Better:

- Bidirectional context provides richer information for prediction
- MLM training exposes the model to diverse masking patterns
- SSM's purely causal nature limits context to left side only

Why Perplexity Differs:

- **SSM:** Causal prediction among $\sim$94K words; higher perplexity indicates difficulty with sequential dependencies
- **BiLSTM:** Bidirectional context among $\sim$94K words; lower perplexity reflects richer context access
- Both models face challenges with the large word vocabulary

Strengths of Each Approach:

- **SSM:** Simpler architecture, faster inference, causal for streaming applications
- **BiLSTM:** Richer context, better for error correction with full context

Limitations:

- High perplexity indicates both models struggle with character patterns
- Limited context window (256 chars) may miss long-range dependencies
- No pretraining or transfer learning used

2.4 Transition to Character-Level for Task 3

Although I initially implemented word-level models, I observed that Task 1 decryption produces many "corrupted" words (e.g., *"investigation"* as *"Ioreena"*). In a word-level model, these are labeled as `<unk>`, making correction impossible. Thus, for the correction task, I also implemented **character-level** variants of Task 2 models. My character-level Bi-LSTM achieved a phenomenally low perplexity of **2.783**, providing a much better foundation for fixing spelling errors.

3 Task 3: Error Correction with Language Models

3.1 Methodology

For this task, I combined the LSTM decryption model with character-level language models from task 2 (had also trained character level LMs along with word level LMs for task 2, both are available on Huggingface).

Why Character-Level? To avoid the massive Unknown Token (<unk>) problem, I used models that operate character by character. This allows the LM to fix individual letters instead of failing to recognize misspelled words. I used a confidence threshold of **0.30**, meaning the LM only intervenes when the decryption model is highly uncertain.

3.2 Results and Evaluation

Since word-level metrics like BLEU and ROUGE evaluate to 0.00 when even a single char is wrong in every word, I used **chrF** (Character n-gram F-score) to measure real structural improvement.

Table 3: Task 3: Error Rectification Performance

Cipher Level	Metric	LSTM Base	+ SSM	+ BiLSTM
Cipher 01 (Low Noise)	Char Accuracy	0.324	0.300	**0.311**
	chrF Score	0.307	0.312	**0.326**
Cipher 02	Char Accuracy	0.246	0.230	**0.237**
	chrF Score	0.288	0.292	**0.300**
Cipher 04 (High Noise)	Char Accuracy	0.167	0.158	**0.161**
	chrF Score	0.267	0.270	**0.275**

3.3 Analysis

- **chrF vs BLEU:** BLEU was 0.00 because sub-word typos prevent perfect word matches. chrF shows that the LMs are successfully making the text more "English-like" by fixing character sequences.
- **Bi-LSTM Advantage:** The Bi-LSTM consistently provided a higher chrF boost (+2.0% on Cipher 01) compared to the SSM, as its bidirectional context is superior for spelling correction.

4 Conclusion

This assignment demonstrated that:

1. **LSTM** handles the numerical substitution cipher much better than a vanilla RNN.

2. **Character-level modeling** is essential for error correction to avoid OOV/Unknown token bottlenecks.

3. **chrF Score** is a more robust metric for noisy decryption than word-level BLEU or ROUGE.

Links to Artifacts

- **HuggingFace Models:** https://huggingface.co/yashmore1224/inlp-final
- **WandB Logs:** https://wandb.ai/yash-more1224-international-institute-of-information-tec/inlp-assignment3?nw=nwuseryashmore1224